

Algorithms and Satisfiability

3. Binary Decision Diagrams

Álvaro Torralba



AALBORG UNIVERSITET

Spring 2025

Agenda

- 1 Introduction
- 2 Binary Decision Diagrams
- 3 Queries
- 4 Operations
- 5 Conclusions

How To Represent Logical Formulas?

Option 1: Logical Formula (e.g. in CNF)

$$(\neg x_1 \vee x_2) \wedge x_3$$

Advantage: Compact representation

Disadvantage: SAT (and other operations) is hard

Option 2: Truth table

x_1	x_2	x_3	value
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	1	0	1
1	1	1	0
1	0	0	0
1	0	1	0

Advantage: SAT is easy (linear in the size of the table)

Disadvantage: Table is **BIG** (exponentially longer than CNF formula)

→ So, does there exist some **compact** representation under which SAT is easy (in the size of the representation)?

Boolean Functions

Definition

A **Boolean** function $f : \{0, 1\}^k \rightarrow \{0, 1\}$ maps k Boolean values to true or false.

x_1	x_2	x_3	value
0	0	0	$1\top$
0	0	1	$0\perp$
0	1	0	$1\top$
0	1	1	$0\perp$
1	1	0	$1\top$
1	1	1	$0\perp$
1	0	0	$0\perp$
1	0	1	$0\perp$

- Notation: We use 0, 1 for the inputs and $\perp$, $\top$ for the output
- Represents sets of assignments to the variables in a logical formula
- Represents a set of sets of elements (over k elements)
 - Example: given k products each with a price, which sets of products can be bought with the available money? which sets of products satisfy our requirements? what's the intersection of the previous sets?

(Reduced Ordered) Binary Decision Diagrams

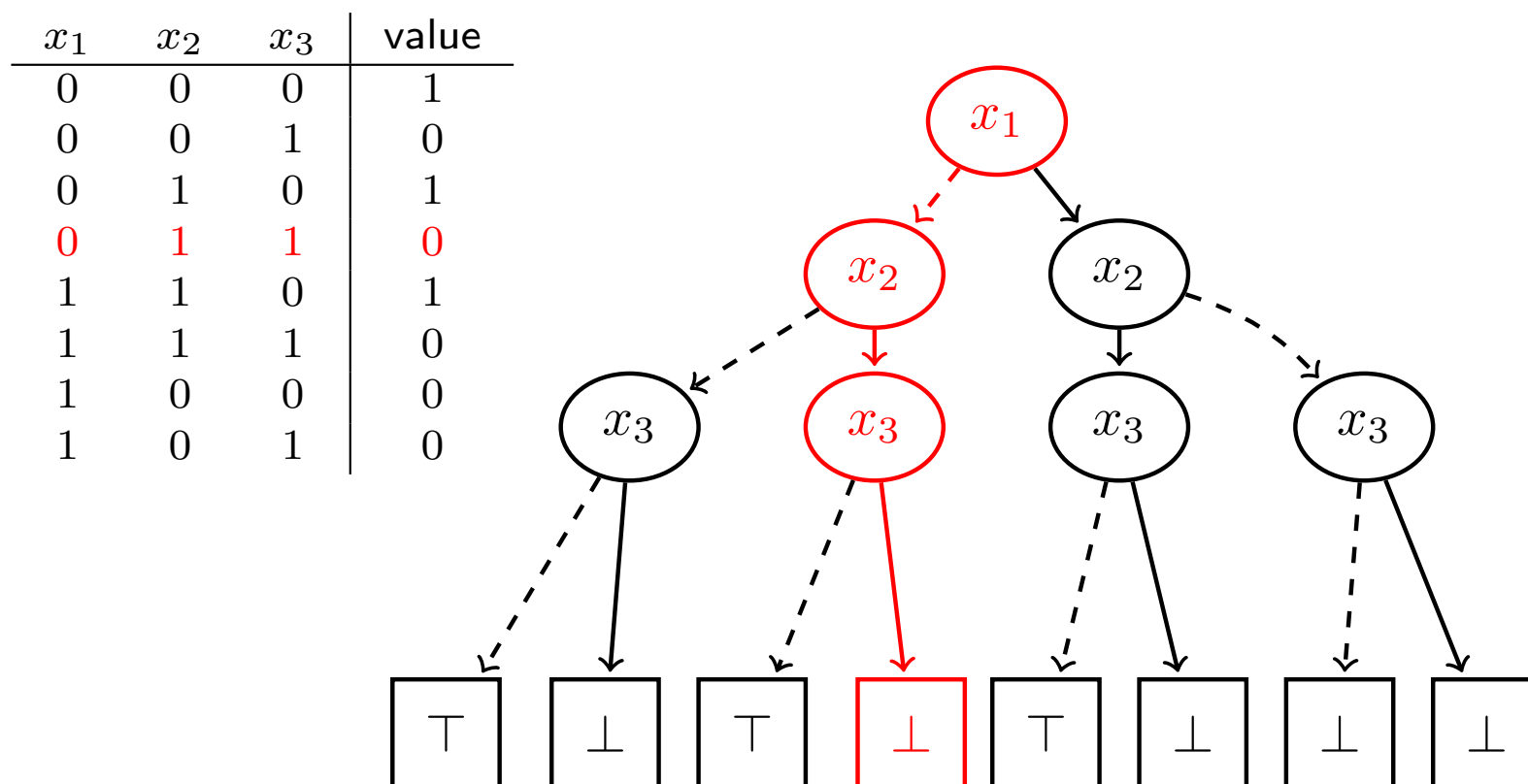
Reduced Ordered Binary Decision Diagrams (or BDDs for short):
representation of logical functions that follows the following key ideas:

- ① **Decision Diagram**: Use Directed Acyclic Graph
- ② **Reduced**: Apply Reduction Rules
- ③ **Ordered**: Fixed Variable Ordering

“BDDs are one of the only really fundamental data structures that came out in the last twenty-five years” — Donald Knuth

Representing Logical Functions as a Decision Tree

Turn a truth table for the Boolean function into a decision tree.



Notation: Nodes are labelled with the variable they checked (x_i), we use solid edge for $x_i = 1$ and dashed edge for $x_i = 0$.

Shannon Expansion

For any Boolean formula f and variable x , it can be written as:

$$f \equiv (\neg x \wedge f[x = 0]) \vee (x \wedge f[x = 1])$$

This is the **Shannon expansion** of f (originally due to G. Boole).

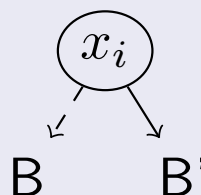
Deja vu? Same principle used by DPLL!

→ Inner nodes correspond to a function represented via the Shannon expansion!

Notation

Node $A = (x_i, B, B')$ represents $f_A = (\overline{x_i} \wedge f_B) \vee (x_i \wedge f_{B'})$

- Variable $v(A) = x_i$ (nodes are labelled with the variable they check)
- Low child $l(A) = B$ (represented by a dashed edge)
- High child $r(A) = B'$ (represented by a solid edge)

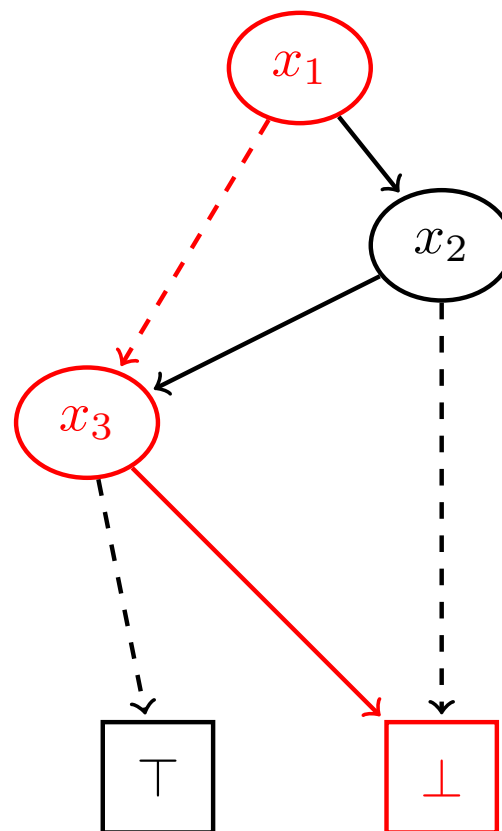


Representing Logical Functions as a Decision Diagram

Diagram: Directed Acyclic Graph (DAG)

→ We can share a node in different parts of the diagram

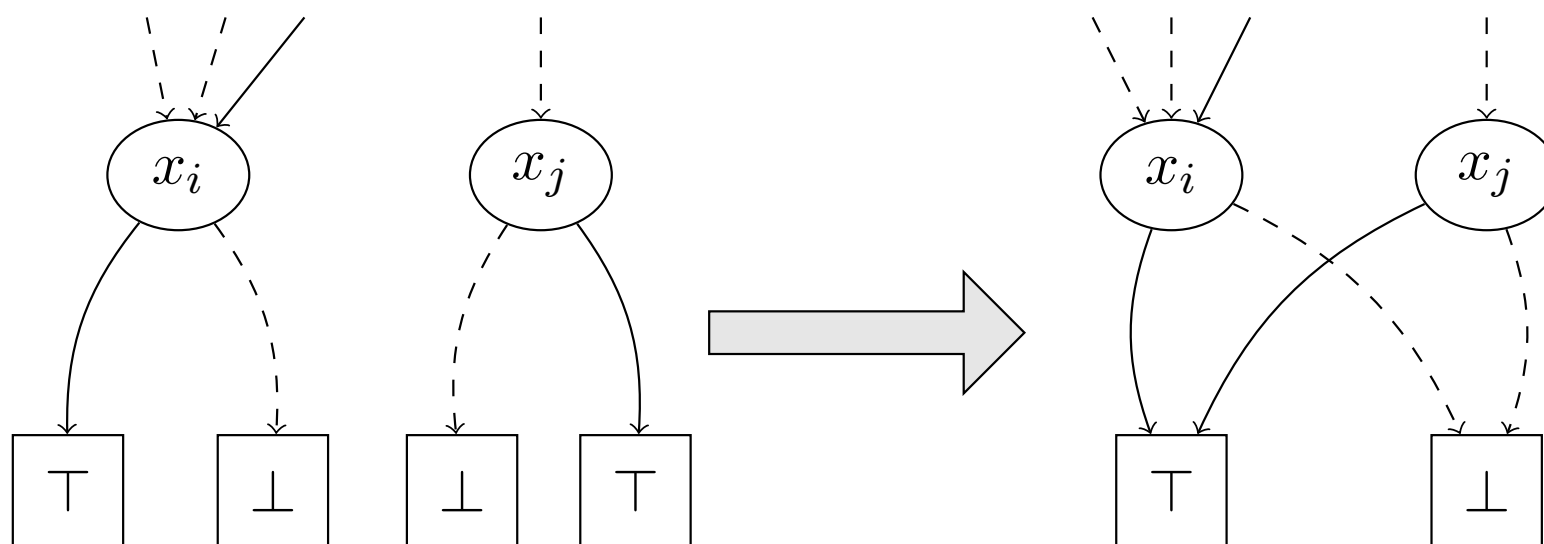
x_1	x_2	x_3	value
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	1	0	1
1	1	1	0
1	0	0	0
1	0	1	0



Notation: We denote $|B|$ to the number of nodes of a DAG B .

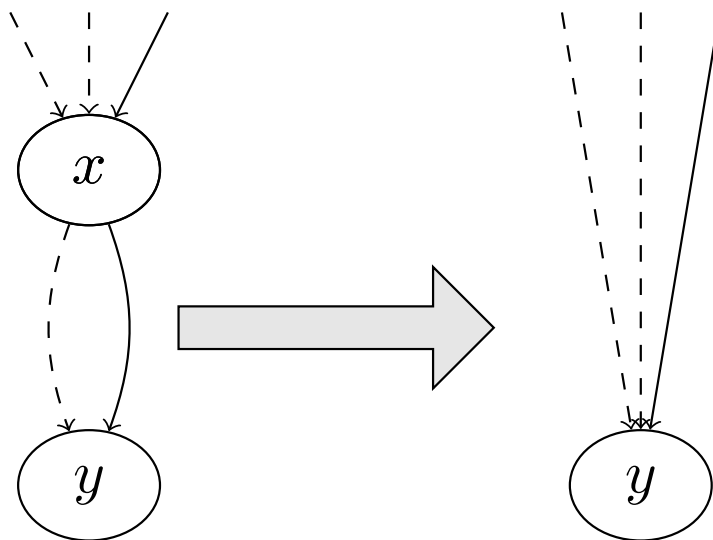
Reduction Rule #1: Eliminate Equivalent Leaves

There are only two unique leaves: $\perp$ and $\top$



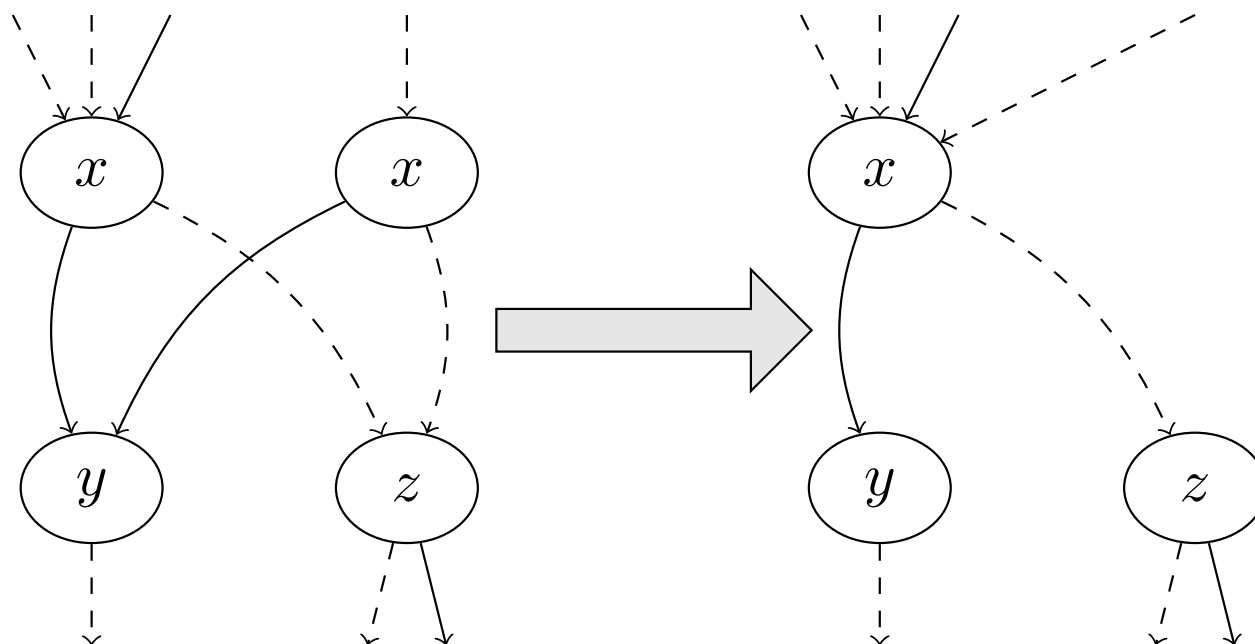
Reduction Rule #2: Eliminate Redundant Nodes

$l(n) = r(n)$, then eliminate n and re-direct all its references to $l(n)$

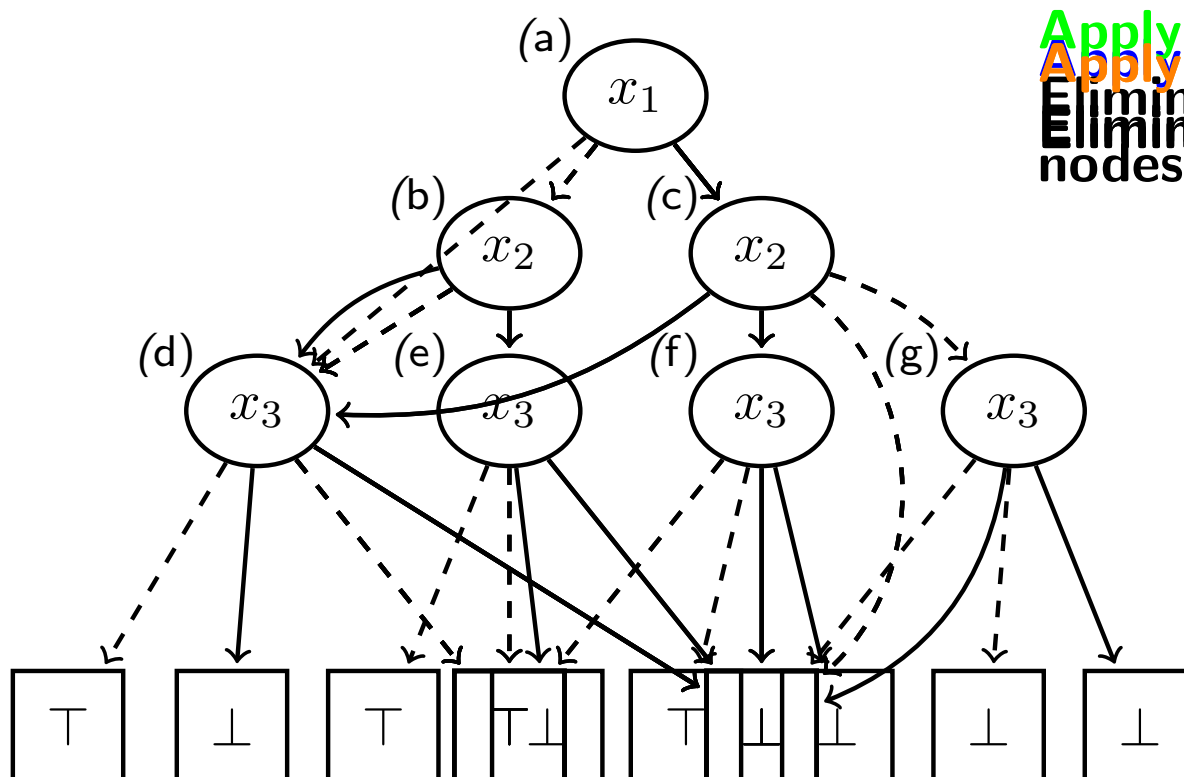


Reduction Rule #3: Eliminate Equivalent Inner Nodes

Two inner nodes n, n' are equivalent if and only if $v(n) = v(n')$, $l(n) = l(n')$, and $r(n) = r(n')$



Reduction Rules Example



Apply reduction rule 3:
Apply reduction rule 2:
Eliminate equivalent inner
nodes
Eliminate redundant nodes

The Reduce Operation: Pseudocode

reduce(A):

```
1 if  $A$  is terminal then
  | // Eliminate equivalent leaves
2   return UniqueTerminal( $A.value$ )
3 if  $A$  in Cache then
4   | return Cache[ $A$ ]
5  $n_l = \text{reduce}(l(A))$ 
6  $n_r = \text{reduce}(r(A))$ 
7 if  $n_l == n_r$  then
  | // Eliminate redundant nodes
8   result =  $n_l$ 
9 else
  | // Eliminate equivalent nodes
10  | result = NodeUnique ( $v(A), n_l, n_r$ )
11 Cache[ $A$ ] = result
12 return result
```

NodeUnique(v, n_l, n_r):

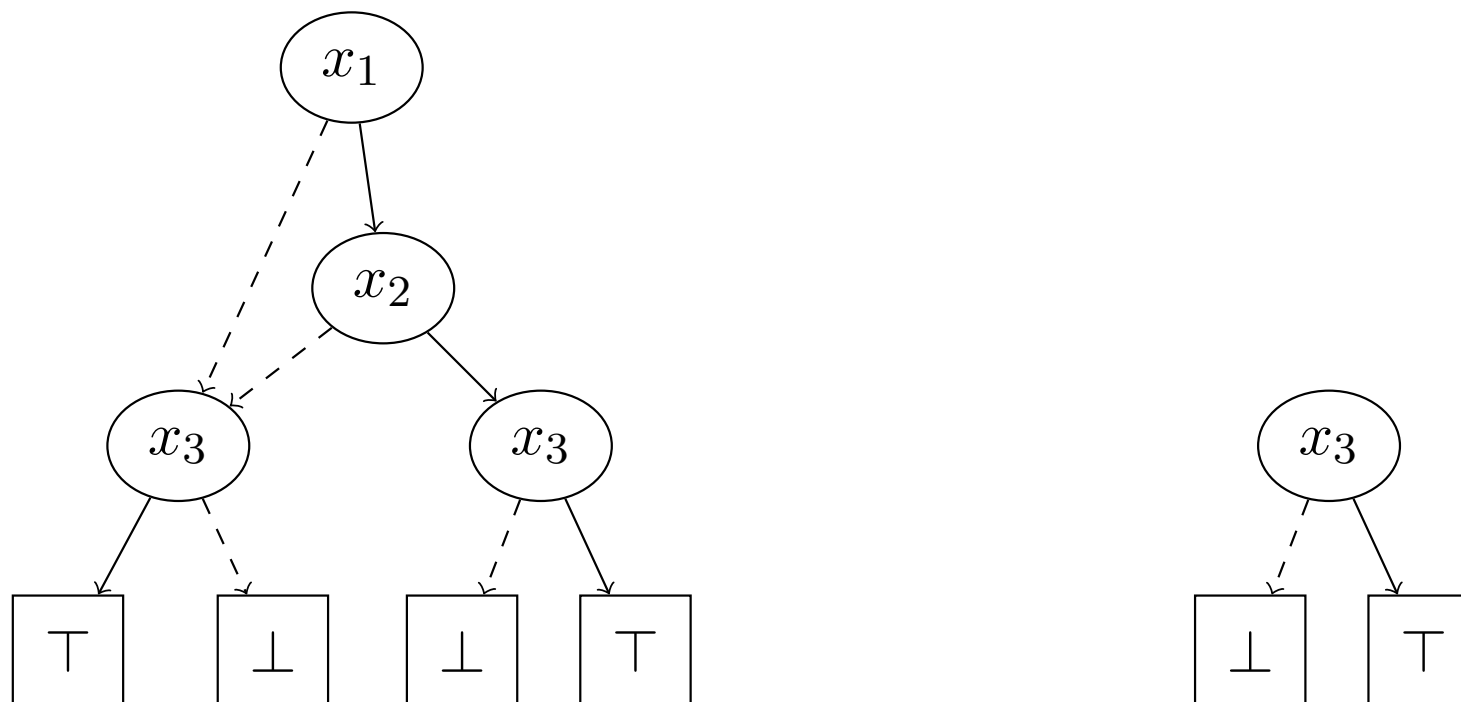
```
1 if ( $v, n_l, n_r$ ) not in GlobalCache
  then
2   | GlobalCache[( $v, n_l, n_r$ )] = new
    | Node ( $v, n_l, n_r$ )
3 return GlobalCache[( $v, n_l, n_r$ )]
```

- GlobalCache is a single cache for all BDDs and operations!
→ Ensures the Equivalence reduction rule

Question: How many calls to **reduce**(A) we perform? $\mathcal{O}(|A|)$: due to the cache, we will never do recursive calls over the same node twice

Overall time complexity: $\mathcal{O}(|A| \log(|A|))$

Questionnaire



Question!

How many nodes would have the resulting BDD after applying Reduce?

(A): 3

(B): 4

(C): 5

(D): 6

Fixed Variable Ordering

Decide arbitrary total order on variables $p_1 < p_2 < \dots < p_n$. Always decompose a function based on the first variable.

Definition (Canonicity). We say that a representation R of a logical formula φ is *canonical* if there is a unique representation:

$$\varphi \equiv \varphi' \implies R(\varphi) = R(\varphi')$$

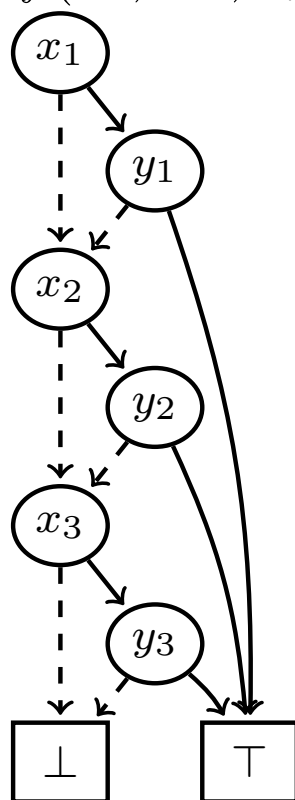
Theorem (ROBDDs are a canonical representation). For a function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ there is exactly one ROBDD u with ordering $p_1 < \dots < p_n$ such that u represents $f(p_1, \dots, p_n)$.

Proof sketch. By induction (base case is trivial for terminal nodes). Assume ROBDDs for functions $f(p_i, \dots, p_n)$ are canonical, then show that ROBDDs for $f(p_{i-1}, \dots, p_n)$ are canonical as well.

Importance of Variable Ordering

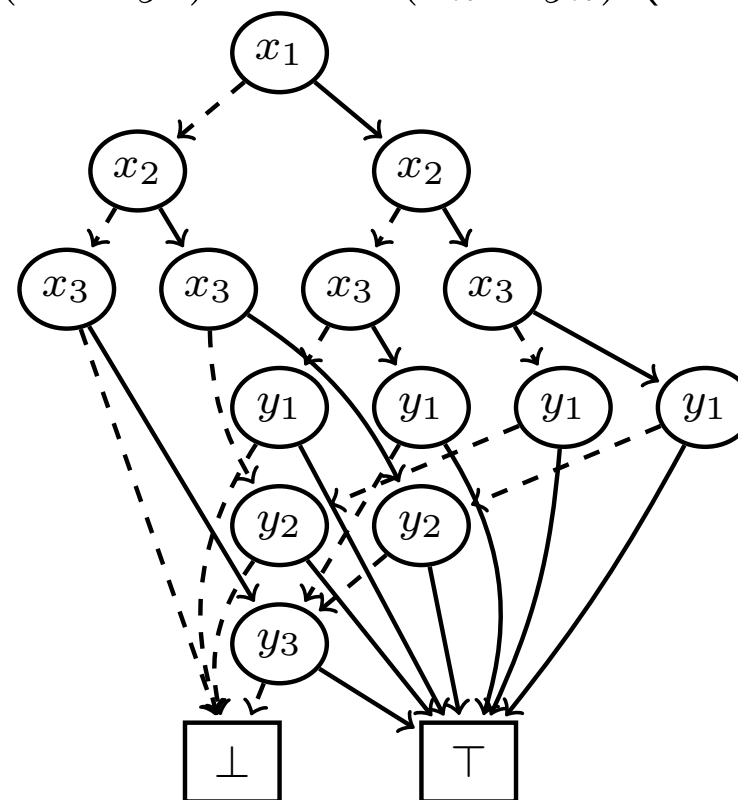
Choice of variable ordering can cause the representation of a function to be exponential or polynomial in the number of variables.

Example: $f(x_1, \dots, x_n, y_1, \dots, y_n) = (x_1 \wedge y_1) \vee \dots \vee (x_n \wedge y_n)$ ($n = 3$).



Polynomial size:

$x_1, y_1, x_2, y_2, x_3, y_3$



Exponential size:

$x_1, x_2, x_3, y_1, y_2, y_3$

Variable Ordering Heuristics

- Static Variable Ordering:
 - Put “related” variables as close as possible in the variable ordering
- Dynamic Variable Ordering:
 - Find the best ordering for a given BDD (or set of BDDs)
 - Optimization problem that is NP-complete
 - But we can just use some approximation

Queries

- **Tautology**: Is $\varphi = \top$?

→ Constant time: just check if $n_\varphi = \top$

- **SAT**: Is $M(\varphi) \neq \emptyset$?

→ Constant time: just check if $n_\varphi \neq \perp$

- **Equivalence** ($\varphi \equiv \psi$): Is $M(\varphi) = M(\psi)$?

→ Constant time: just check if $n_\varphi = n_\psi$

- **Model counting**: Number of satisfying assignments: $|M(\varphi)|$

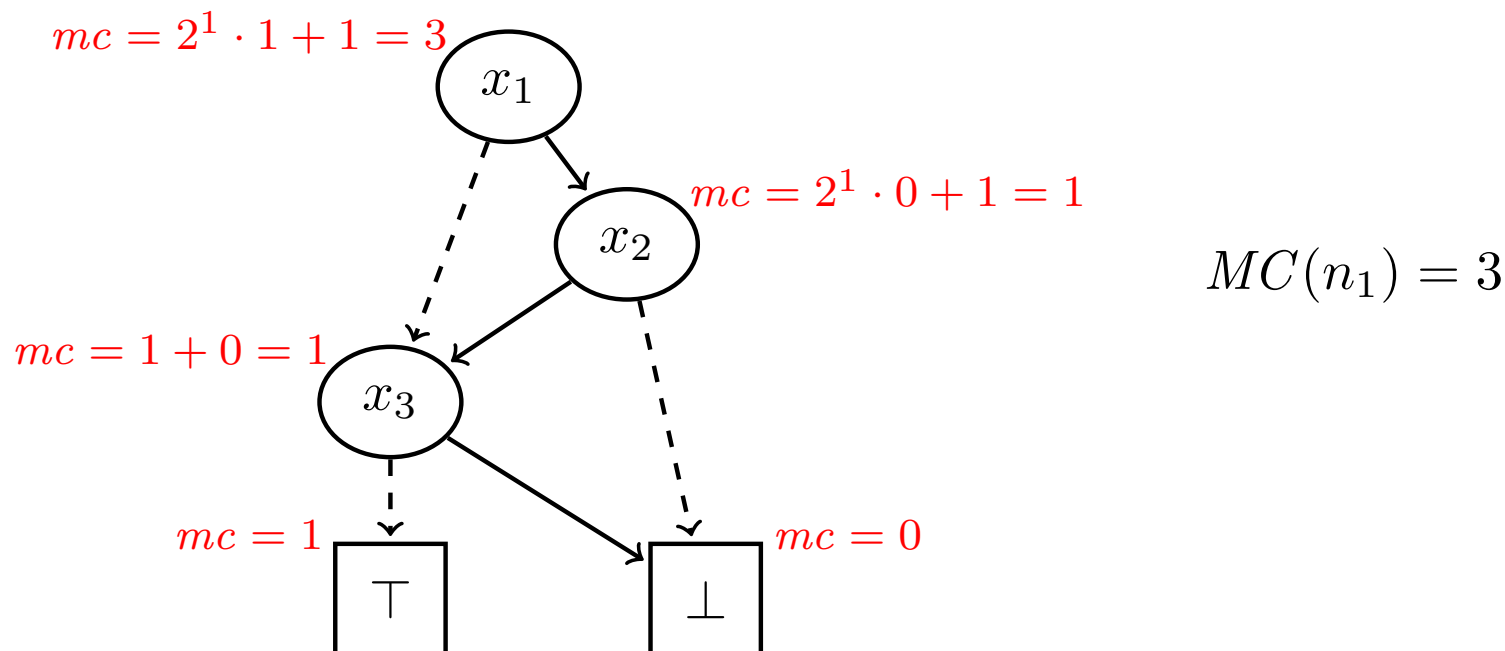
→ Linear time: dynamic programming algorithm

Model Counting: Example

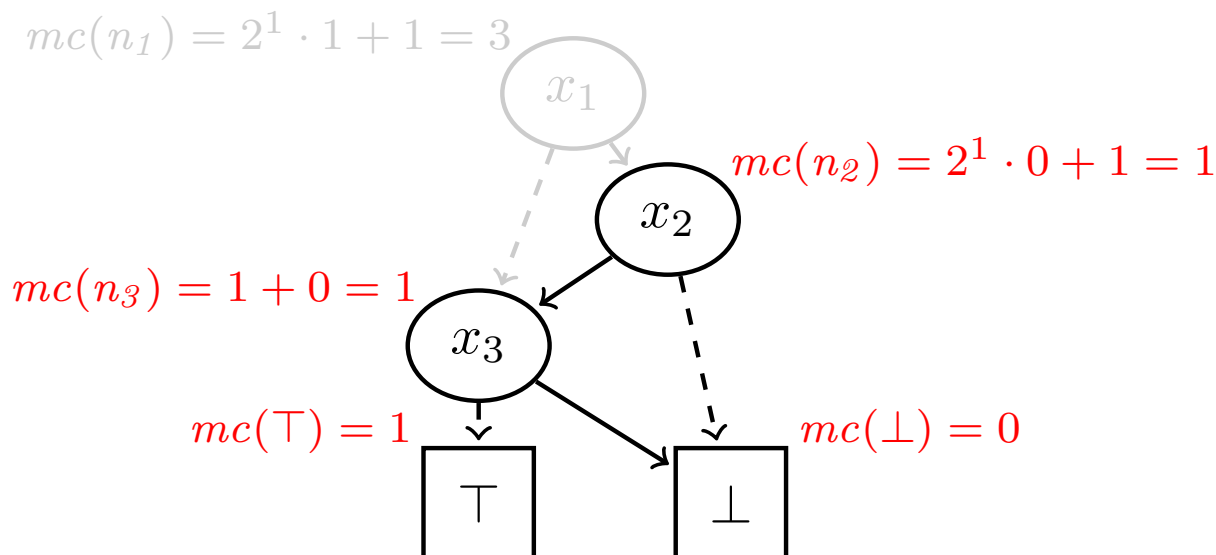
$MC(f)$: number of assignments over $x_1, \dots, x_n$ that make f true.

$MC(f) = 2^{v(f)} mc(f)$ where:

- Leaf nodes: $mc(\top) = 1, mc(\perp) = 0$ and define $v(\perp) = v(\top) = x_{n+1}$
- Inner nodes: $mc(f) = 2^{j-i-1} \cdot mc(f_0) + 2^{k-i-1} \cdot mc(f_1)$ where $f = (x_i, f_0, f_1)$, $v(f_0) = x_j$, $v(f_1) = x_k$, then



Questionnaire



Question!

How many models does n_2 have over $x_1, \dots, x_3$? ($MC(n_2)$)

(A): 1

(B): 2

(C): 3

(D): 4

2 assignments (010 and 110), so $MC(n_2) = 2$.

Similarly, $MC(n_3) = 2^2 \cdot 1 = 4$, $MC(\top) = 2^3 \cdot 1 = 8$

→ We must specify over how many variables we perform model counting

BDD Operations

- **Reduce** (f)
→ Reduces the canonical form of a function f represented as a non-reduced BDD
- **Apply** (f_1, f_2)
→ Generic implementation for binary operations ($\vee, \wedge, \dots$)
- **If-Then-Else (ITE)** (f_1, f_2, f_3)
→ Generic implementation for ternary operations
- **Exists** (X, f)
→ Forgets variables X from the BDD

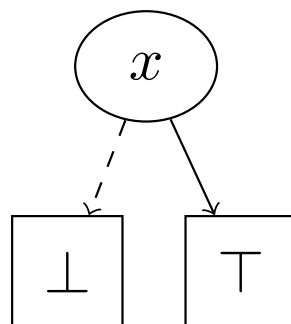
Keys:

- Recursively iterate over the BDD
- Dynamic programming: Use cache to store intermediate results. Never re-do the same work twice!

But... how to obtain a BDD in the first place?

BDDs are always reduced (the reduce function is never actually used!)

You can easily obtain the BDDs that represent simple functions:



And then, use operations on them to construct more complex functions!

$$(\neg x_1 \vee x_2) \wedge x_3$$

The Apply Operation

Given compatible OBDDs B_f and B_g that represent formulas f and g , $\text{apply}(\square, B_f, B_g)$ computes an OBDD representing $f \square g$.

- Compatible: **operands must use the same variable ordering**
- $\square$ is some binary operation on Boolean formulas (e.g. $\vee, \wedge, \oplus$)
- Unary operations can be handled too.
for example, negation: $\neg x = x \oplus 1$

Using the Shannon expansion, $f \square g$ can be expanded as:

$$f \square g \equiv (\neg x \wedge (f[x=0] \square g[x=0])) \vee (x \wedge (f[x=1] \square g[x=1]))$$

The Apply Operation: Pseudocode

apply($\square$, A , B):

```
1  if  $A$  is terminal and  $B$  is terminal then
2    └ return UniqueTerminal( $A.value \square B.value$ )
3  if  $(A, B)$  in Cache then
4    └ return Cache[ $(A, B)$ ]
5  if  $v(A) == v(B)$  then
6    └  $v = v(A)$ 
7    └  $n_l = \textit{apply}(\square, l(A), l(B))$ 
8    └  $n_r = \textit{apply}(\square, r(A), r(B))$ 
9  else if  $B$  is terminal or  $v(B) > v(A)$  then
10   └  $v = v(A)$ 
11   └  $n_l = \textit{apply}(\square, l(A), B)$ 
12   └  $n_r = \textit{apply}(\square, r(A), B)$ 
13 else
14   └  $v = v(B)$ 
15   └  $n_l = \textit{apply}(\square, A, l(B))$ 
16   └  $n_r = \textit{apply}(\square, A, r(B))$ 
17 result = NodeUnique ( $v, n_l, n_r$ )
18 Cache[ $(A, B)$ ] = result
19 return result
```


The Apply Operation: Cases

$$1 \quad \text{apply}(\square, \boxed{u}, \boxed{v}) = \boxed{u \square v}$$

$$2 \quad \text{apply}(\square, \begin{array}{c} \textcircled{x} \\ / \quad \backslash \\ B \quad B' \end{array}, \begin{array}{c} \textcircled{x} \\ / \quad \backslash \\ C \quad C' \end{array}) = \begin{array}{c} \textcircled{x} \\ / \quad \backslash \\ \text{apply}(\square, B, C) \quad \text{apply}(\square, B', C') \end{array}$$

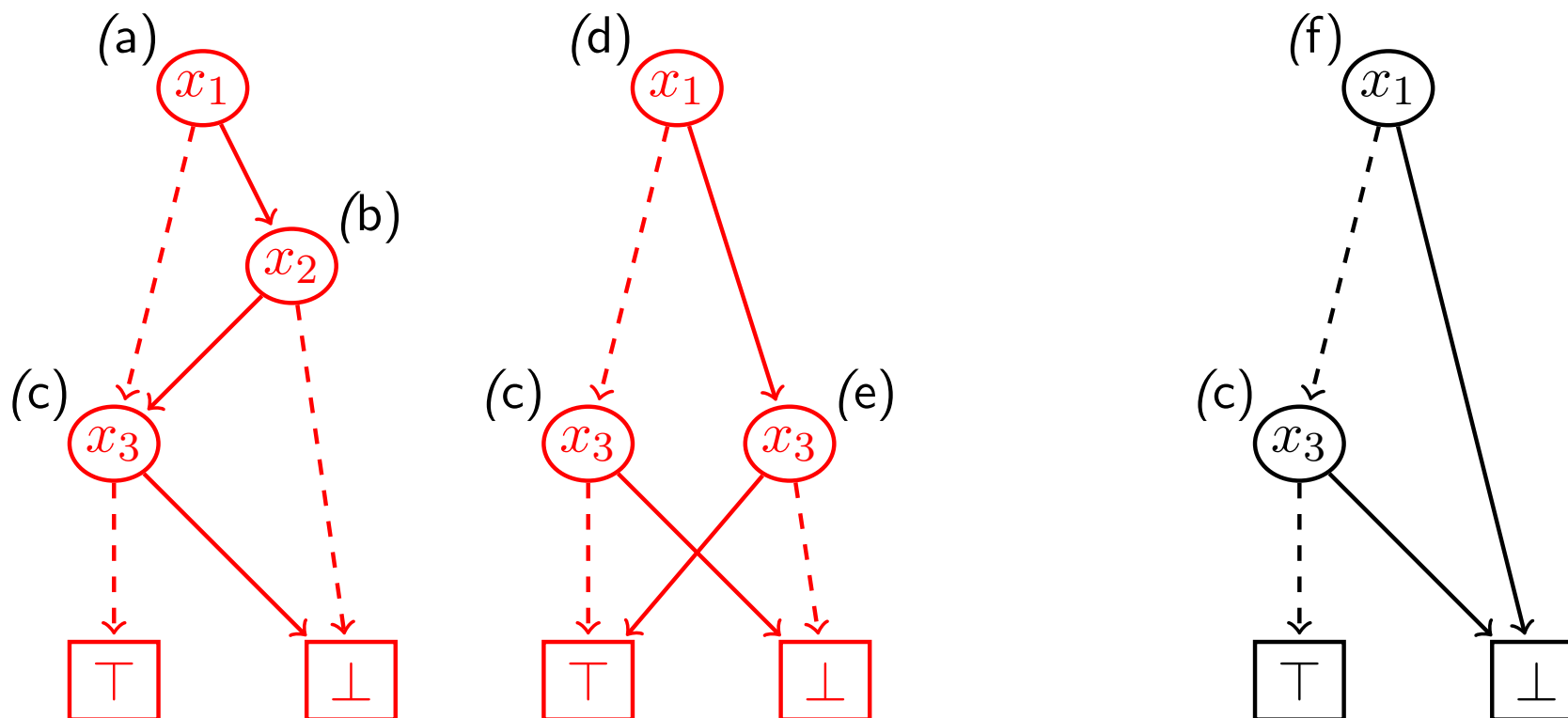
$$3 \quad \text{apply}(\square, \begin{array}{c} \textcircled{x} \\ / \quad \backslash \\ B \quad B' \end{array}, C) = \begin{array}{c} \textcircled{x} \\ / \quad \backslash \\ \text{apply}(\square, B, C) \quad \text{apply}(\square, B', C) \end{array}$$

when C is terminal node, or non-terminal with $v(C) > x$

$$4 \quad \text{apply}(\square, B, \begin{array}{c} \textcircled{x} \\ / \quad \backslash \\ C \quad C' \end{array}) = \begin{array}{c} \textcircled{x} \\ / \quad \backslash \\ \text{apply}(\square, B, C) \quad \text{apply}(\square, B, C') \end{array}$$

when B is terminal node, or non-terminal with $v(B) > x$

The Apply Operation: Example (Conjunction)



1. $(a, d) \mapsto (x_1, \neg c, \neg \perp) \mapsto f$
2. $(c, c) \mapsto (x_3, \neg \top, \neg \perp) \mapsto c$
3. $(\top, \top) \mapsto \top$
4. $(\perp, \perp) \mapsto \perp$
5. $(b, e) \mapsto (x_2, \neg \perp, \neg \perp) \mapsto \perp$

6. $(\perp, e) \mapsto \perp$
7. $(\perp, \perp) \mapsto \perp$
8. $(c, e) \mapsto (x_3, \neg \perp, \neg \perp) \mapsto \perp$
9. $(\top, \perp) \mapsto \perp$
10. $(\perp, \top) \mapsto \perp$

The Apply Operation: Running Time

Question!

What is the running time of $\text{apply}(\square, B_f, B_g)$?

Answer: Quadratic, $\mathcal{O}(|B_f||B_g|)$. The number of calls to the recursive function in which we do not return immediately is bounded by the pairs of nodes, because we cache the results for every pair of input nodes.

Question!

What is the maximum size of the BDD representing $f \wedge g$?

Answer: $|B_f||B_g|$, same reason as above.

Question!

So, is the BDD representing each CNF formula of polynomial size?

Answer: NO! If the CNF formula has N clauses, each represented with a BDD with K nodes then in the worst case we have K^N nodes.

Complexity of Operations (extracted from Darwiche and Marquis (2002))

L	Queries				Transformations				
	CO	VA	EQ	CT	$\bigwedge_n$	$\bigwedge$	$\bigvee_n$	$\bigvee$	$\neg C$
NNF	○	○	○	○	✓	✓	✓	✓	✓
OBDD $\leq$	✓	✓	✓	✓	●	✓	●	✓	✓
DNF	✓	○	○	○	●	✓	✓	✓	●
CNF	○	✓	○	○	✓	✓	●	✓	●

✓: Polynomial Time, ● (○) Exponential time (unless P=NP)

Summary

Reduced Ordered Binary Decision Diagrams (BDDs for short)

- Directed acyclic graph with two leave nodes $\perp$, $\top$
- Each node decomposes the function using the [Shannon expansion](#)
- [Reduced](#)
any node with two identical children is removed
equivalent nodes are merged
- [Ordered](#)
Splitting variables always follow the same order along all paths
 $x_1 < x_2 < x_3 < \dots < x_n$
- Properties:
 - Canonical representation
 - Efficient queries
 - Efficient operations

Some Caveats

We made some simplifying assumptions, and didn't mention a lot of optimizations. For example:

- Typically, negation is encoded on the edges
 - The same node is used to represent a function and its negation
 - Negation can be performed in constant time
 - One needs to handle this during the apply operation
- In our version of apply we always reach the terminals. However, we can return early in some cases by encoding some additional if conditions
 - $f \vee f = f$
 - $f \vee \perp = f$
 - $f \vee \top = \top$
 - $f \wedge f = f$
 - ...

Some Extensions

- What if instead of two terminals ($\perp, \top$), I have many (e.g. Numbers)
→ [Algebraic Decision Diagrams \(ADD\)](#)
(you can also use multiple BDDs or auxiliary variables in the BDD)
- What if my variables are not Boolean but multi-valued?
→ [Multi-valued decision diagrams \(MDDs\)](#)
(you can also use as auxiliary variables in the BDD)
- Are there other possible reduction rules?
→ [Zero-Suppressed Decision Diagrams \(ZDDs\)](#)
- Can I generalize Shannon Expansion?
→ [Sentential Decision Diagrams \(SDDs\)](#)
- Can I represent more complex arithmetic expressions?
→ [Edge-valued Multi-valued Decision Diagrams \(EVMDDs\)](#)

Reading

- The art of Computer Programming. Knuth. Section 7.1.4. Binary Decision Diagrams
- Graph-based algorithms for Boolean function manipulation [Bryant (1986)].
→Original paper exploring BDDs
- A Knowledge Compilation Map [Darwiche and Marquis (2002)].
→A very good summary of how BDDs relate to other alternative representations

References I

Randal E. Bryant. Graph-based algorithms for boolean function manipulation. *IEEE Transactions on Computers*, 35(8):677–691, 1986.

Adnan Darwiche and Pierre Marquis. A knowledge compilation map. *J. Artif. Intell. Res.*, 17:229–264, 2002.